# Kotlin coroutines on Android

June 16, 2026 5:00 PM

https://developer.android.com/kotlin/coroutines





## Kotlin coroutines on Android

A *coroutine* is a concurrency design pattern that you can use on Android to simplify code that executes asynchronously. Coroutines were added to Kotlin in version 1.3 and are based on established concepts from other languages.

On Android, coroutines help to manage long-running tasks that might otherwise block the main thread and cause your app to become unresponsive. Over 50% of professional developers who use coroutines have reported seeing increased productivity. This topic describes how you can use Kotlin coroutines to address these problems, enabling you to write cleaner and more concise app code.

### Features

Coroutines is our recommended solution for asynchronous programming on Android. Noteworthy features include the following:

- **Lightweight**: You can run many coroutines on a single thread due to support for *suspension*, which doesn't block the thread where the coroutine is running. Suspending saves memory over blocking while supporting many concurrent operations.
- **Fewer memory leaks**: Use *structured concurrency* to run operations within a scope.
- **Built-in cancellation support**: Cancellation is propagated automatically through the running coroutine hierarchy.
- **Jetpack integration**: Many Jetpack libraries include extensions that provide full coroutines support. Some libraries also provide their own coroutine scope that you can use for structured concurrency.

### Examples overview

Based on the Guide to app architecture, the examples in this topic make a network request and return the result to the main thread, where the app can then display the result to the user.

Specifically, the `ViewModel` Architecture component calls the repository layer on the main thread to trigger the network request. This guide iterates through various solutions that use coroutines to keep the main thread unblocked

the main thread, where the app can then display the result to the user.

Specifically, the `ViewModel` Architecture component calls the repository layer on the main thread to trigger the network request. This guide iterates through various solutions that use coroutines to keep the main thread unblocked.

`ViewModel` includes a set of KTX extensions that work directly with coroutines. These extension are `lifecycle-viewmodel-ktx` library and are used in this guide.

## Dependency info

To use coroutines in your Android project, add the following dependency to your app's `build.gradle` file:

Groovy Kotlin

```
dependencies {
    implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-android:1.3.9'
}
```

## Executing in a background thread

Making a network request on the main thread causes it to wait, or *block*, until it receives a response. Since the thread is blocked, the OS isn't able to call `onDraw()`, which causes your app to freeze and potentially leads to an Application Not Responding (ANR) dialog. For a better user experience, let's run this operation on a background thread.

First, let's take a look at our `Repository` class and see how it's making the network request:

```
sealed class Result<out R> {
    data class Success<out T>(val data: T) : Result<T>()
    data class Error(val exception: Exception) : Result<Nothing>()
}

class LoginRepository(private val responseParser: LoginResponseParser) {
    private const val loginUrl = "https://example.com/login"

    // Function that makes the network request, blocking the current thread
    fun makeLoginRequest(
        jsonBody: String
    ): Result<LoginResponse> {
        val url = URL(loginUrl)
        (url.openConnection() as? HttpURLConnection)?.run {
            requestMethod = "POST"
            setRequestProperty("Content-Type", "application/json; utf-8")
            setRequestProperty("Accept", "application/json")
            doOutput = true
            outputStream.write(jsonBody.toByteArray())
            return Result.Success(responseParser.parse(inputStream))
        }
        return Result.Error(Exception("Cannot open HttpURLConnection"))
    }
}
```

`makeLoginRequest` is synchronous and blocks the calling thread. To model the response of the network request, we have our own `Result` class.

The `ViewModel` triggers the network request when the user clicks, for example, on a button:

```
class LoginViewModel(
    private val loginRepository: LoginRepository
): ViewModel() {

    fun login(username: String, token: String) {
        val jsonBody = "{ username: \"$username\", token: \"$token\"}"
        loginRepository.makeLoginRequest(jsonBody)
```

```
    fun login(username: String, token: String) {
        val jsonBody = "{ username: \"$username\", token: \"$token\"}"
        loginRepository.makeLoginRequest(jsonBody)
    }
}
```

With the previous code, `LoginViewModel` is blocking the UI thread when making the network request. The simplest solution to move the execution off the main thread is to create a new coroutine and execute the network request on an I/O thread:

```
class LoginViewModel(
    private val loginRepository: LoginRepository
): ViewModel() {

    fun login(username: String, token: String) {
        // Create a new coroutine to move the execution off the UI thread
        viewModelScope.launch(Dispatchers.IO) {
            val jsonBody = "{ username: \"$username\", token: \"$token\"}"
            loginRepository.makeLoginRequest(jsonBody)
        }
    }
}
```

Let's dissect the coroutines code in the `login` function:

- `viewModelScope` is a predefined `CoroutineScope` that is included with the `ViewModel` KTX extensions. Note that all coroutines must run in a scope. A `CoroutineScope` manages one or more related coroutines.
- `launch` is a function that creates a coroutine and dispatches the execution of its function body to the corresponding dispatcher.
- `Dispatchers.IO` indicates that this coroutine should be executed on a thread reserved for I/O operations.

The `login` function is executed as follows:

- The app calls the `login` function from the `View` layer on the main thread.
- `launch` creates a new coroutine, and the network request is made independently on a thread reserved for I/O operations.
- While the coroutine is running, the `login` function continues execution and returns, possibly before the network request is finished. Note that for simplicity, the network response is ignored for now.

Since this coroutine is started with `viewModelScope`, it is executed in the scope of the `ViewModel`. If the `ViewModel` is destroyed because the user is navigating away from the screen, `viewModelScope` is automatically cancelled, and all running coroutines are canceled as well.

One issue with the previous example is that anything calling `makeLoginRequest` needs to remember to explicitly move the execution off the main thread. Let's see how we can modify the `Repository` to solve this problem for us.

## Use coroutines for main-safety

We consider a function *main-safe* when it doesn't block UI updates on the main thread. The `makeLoginRequest` function is not main-safe, as calling `makeLoginRequest` from the main thread does block the UI. Use the `withContext()` function from the coroutines library to move the execution of a coroutine to a different thread:

```
class LoginRepository(...) {
    ...
    suspend fun makeLoginRequest(
        jsonBody: String
    ): Result<LoginResponse> {

        // Move the execution of the coroutine to the I/O dispatcher
        return withContext(Dispatchers.IO) {
```

```
        // Move the execution of the coroutine to the I/O dispatcher
        return withContext(Dispatchers.IO) {
            // Blocking network request code
        }
    }
}
```

`withContext(Dispatchers.IO)` moves the execution of the coroutine to an I/O thread, making our calling function main-safe and enabling the UI to update as needed.

`makeLoginRequest` is also marked with the `suspend` keyword. This keyword is Kotlin's way to enforce a function to be called from within a coroutine.

> ★ **Note:** For easier testing, we recommend injecting **`Dispatchers`** into a **`Repository`** layer. To learn more, see Testing coroutines on Android.

In the following example, the coroutine is created in the `LoginViewModel`. As `makeLoginRequest` moves the execution off the main thread, the coroutine in the `login` function can be now executed in the main thread:

```
class LoginViewModel(
    private val loginRepository: LoginRepository
): ViewModel() {

    fun login(username: String, token: String) {

        // Create a new coroutine on the UI thread
        viewModelScope.launch {
            val jsonBody = "{ username: \"$username\", token: \"$token\"}"

            // Make the network call and suspend execution until it finishes
            val result = loginRepository.makeLoginRequest(jsonBody)

            // Display result of the network request to the user
            when (result) {
                is Result.Success<LoginResponse> -> // Happy path
                else -> // Show error in UI
            }
        }
    }
}
```

Note that the coroutine is still needed here, since `makeLoginRequest` is a `suspend` function, and all `suspend` functions must be executed in a coroutine.

This code differs from the previous `login` example in a couple of ways:

- `launch` doesn't take a `Dispatchers.IO` parameter. When you don't pass a `Dispatcher` to `launch`, any coroutines launched from `viewModelScope` run in the main thread.
- The result of the network request is now handled to display the success or failure UI.

The login function now executes as follows:

- The app calls the `login()` function from the `View` layer on the main thread.
- `launch` creates a new coroutine on the main thread, and the coroutine begins execution.
- Within the coroutine, the call to `loginRepository.makeLoginRequest()` now *suspends* further execution of the coroutine until the `withContext` block in `makeLoginRequest()` finishes running.
- Once the `withContext` block finishes, the coroutine in `login()` resumes execution *on the main thread* with the result of the network request.

> ★ **Note:** To communicate with the **`View`** from the **`ViewModel`** layer, use **`LiveData`** as recommended in the Guide to app architecture. When following this pattern, the code in the **`ViewModel`** is executed on the main thread, so you can call

**Note:** To communicate with the `View` from the `ViewModel` layer, use `LiveData` as recommended in the Guide to app architecture. When following this pattern, the code in the `ViewModel` is executed on the main thread, so you can call `MutableLiveData`'s `setValue()` function directly.

## Handling exceptions

To handle exceptions that the `Repository` layer can throw, use Kotlin's built-in support for exceptions. In the following example, we use a `try-catch` block:

```
class LoginViewModel(
    private val loginRepository: LoginRepository
): ViewModel() {

    fun login(username: String, token: String) {
        viewModelScope.launch {
            val jsonBody = "{ username: \"$username\", token: \"$token\"}"
            val result = try {
                loginRepository.makeLoginRequest(jsonBody)
            } catch(e: Exception) {
                Result.Error(Exception("Network request failed"))
            }
            when (result) {
                is Result.Success<LoginResponse> -> // Happy path
                else -> // Show error in UI
            }
        }
    }
}
```

In this example, any unexpected exception thrown by the `makeLoginRequest()` call is handled as an error in the UI.

## Additional coroutines resources

For a more detailed look at coroutines on Android, see Improve app performance with Kotlin coroutines.

For more coroutines resources, see the following links:

- Coroutines overview (JetBrains)
- Coroutines guide (JetBrains)
- Additional resources for Kotlin coroutines and flow

Was this helpful?